

Отчет по третьему этапу внешнего курса

Продвинутые темы

Виноградова Мария Андреевна

Содержание

1	Цель работы	6
2	Выполнение третьего этапа внешнего курса	7
3	Выводы	44

Список иллюстраций

2.1	Смотрим текст вопроса	7
2.2	Изучаем краткую справку программы	8
2.3	Смотрим текст вопроса	9
2.4	Практически проверяем каждый ответ	10
2.5	Смотрим текст вопроса	11
2.6	Практически проверяем каждый ответ	12
2.7	Смотрим текст вопроса	13
2.8	Пишем команду	14
2.9	Смотрим текст вопроса	15
2.10	Смотрим текст вопроса	16
2.11	Проверяем историю команд после запуска 3 команд	17
2.12	Смотрим текст вопроса	18
2.13	Проверяем историю команд после запуска 3 команд	18
2.14	Смотрим текст вопроса	19
2.15	Смотрим на вывод программы	19
2.16	Смотрим текст вопроса	20
2.17	Пишем программу	20
2.18	Сверяем вывод и пример	21
2.19	Смотрим текст вопроса	21
2.20	Пишем программу	22
2.21	Смотрим вывод	22
2.22	Смотрим текст вопроса	23
2.23	Смотрим текст вопроса	23
2.24	Пишем программу	24
2.25	Сверяем вывод и пример	24
2.26	Смотрим текст вопроса	25
2.27	Смотрим текст вопроса	25
2.28	Пишем программу	26
2.29	Сверяем вывод и пример	27
2.30	Смотрим текст вопроса	28
2.31	Пишем программу	29
2.32	Сверяем вывод и пример	30
2.33	Смотрим текст вопроса	30
2.34	Смотрим текст вопроса	31
2.35	Пишем программу	32
2.36	Проверяем вывод	33
2.37	Смотрим текст вопроса	34

2.38 Пишем программу	34
2.39 Сверяем вывод и пример	35
2.40 Смотрим текст вопроса	35
2.41 Пишем программу	36
2.42 Сверяем вывод и пример	37
2.43 Смотрим текст вопроса	37
2.44 Смотрим текст вопроса	38
2.45 Смотрим текст вопроса	39
2.46 Смотрим текст вопроса	39
2.47 Смотрим текст вопроса	40
2.48 Практически проверяем каждый ответ	41
2.49 Смотрим текст вопроса	41
2.50 Смотрим текст вопроса	42
2.51 Создаем файлы и заполняем один из них	42
2.52 Пишем инструкцию	43
2.53 Проверяем результат	43

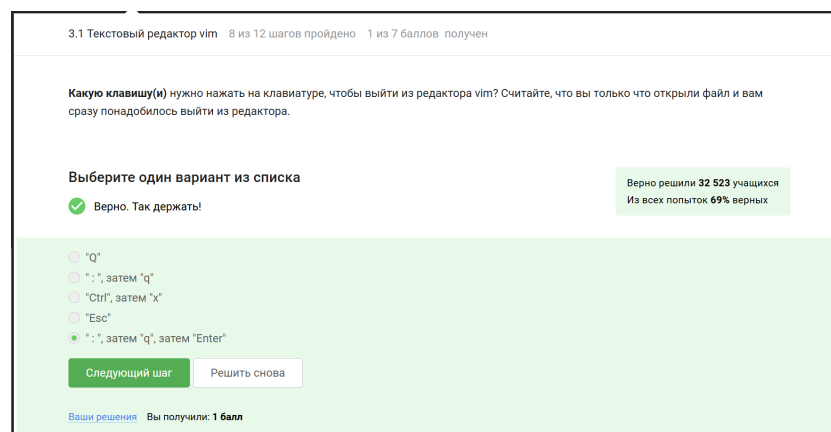
Список таблиц

1 Цель работы

Рассмотреть несколько продвинутых тем, включающих в себя основы написания скриптов на языке `bash`, использовать многофункционального текстового редактора `vim` и другие.

2 Выполнение третьего этапа внешнего курса

Ознакамливаемся с первым вопросом (рис. 2.1).



3.1 Текстовый редактор vim 8 из 12 шагов пройдено 1 из 7 баллов получен

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

☒ Верно. Так держать!

Верно решили 32 523 учащихся
Из всех попыток 69% верных

- ☐ "q"
- ☐ ":", затем "q"
- ☐ "Ctrl", затем "x"
- ☐ "Esc"
- ☒ ":", затем "q", затем "Enter"

[Ваше решение](#) Вы получили: 1 балл

Следующий шаг Решить снова

Рис. 2.1: Смотрим текст вопроса

Почему ответ верен:

Запускаем из консоли приложение tmux, предварительно его скачав и сразу видим краткую справку по программе из которой и узнаем ответ (рис. 2.2).

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку.

```
Strange_ TEXT is_here. 2=2 YES!
```

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать **vim-справку** по, например, перемещению `w`, нужно открыть vim и ввести команду `:help w`. Вы попадете в то место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про `e` и про `b` и, самое главное, про `word` и `WORD`. Кроме того, можно вызвать сразу справку по термину `word` при помощи `:help word`. Чтобы закрыть справку, нужно ввести команду `:q`.

Выберите все подходящие ответы из списка

Верно решили **25 385** учащихся
Из всех попыток **20%** верных

✓ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Чтобы попасть в конец строки, нужно совершить больше нажатий на W, чем на w
- ☒ Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w
- ☐ Чтобы попасть в конец строки, нужно одинаковое число нажатий, что на W, что на w
- ☒ Нажимая только на W, нельзя переместить курсор на "."
- ☒ В этой строке 9 "слов" (word)
- ☒ После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w

Следующий шаг Решить снова

[Ваши решения](#) Вы получили: **1 балл**

Рис. 2.3: Смотрим текст вопроса

Почему ответ верен:

Запускаем tmux, на практике и проверяем все варианты ответа (рис. 2.4).

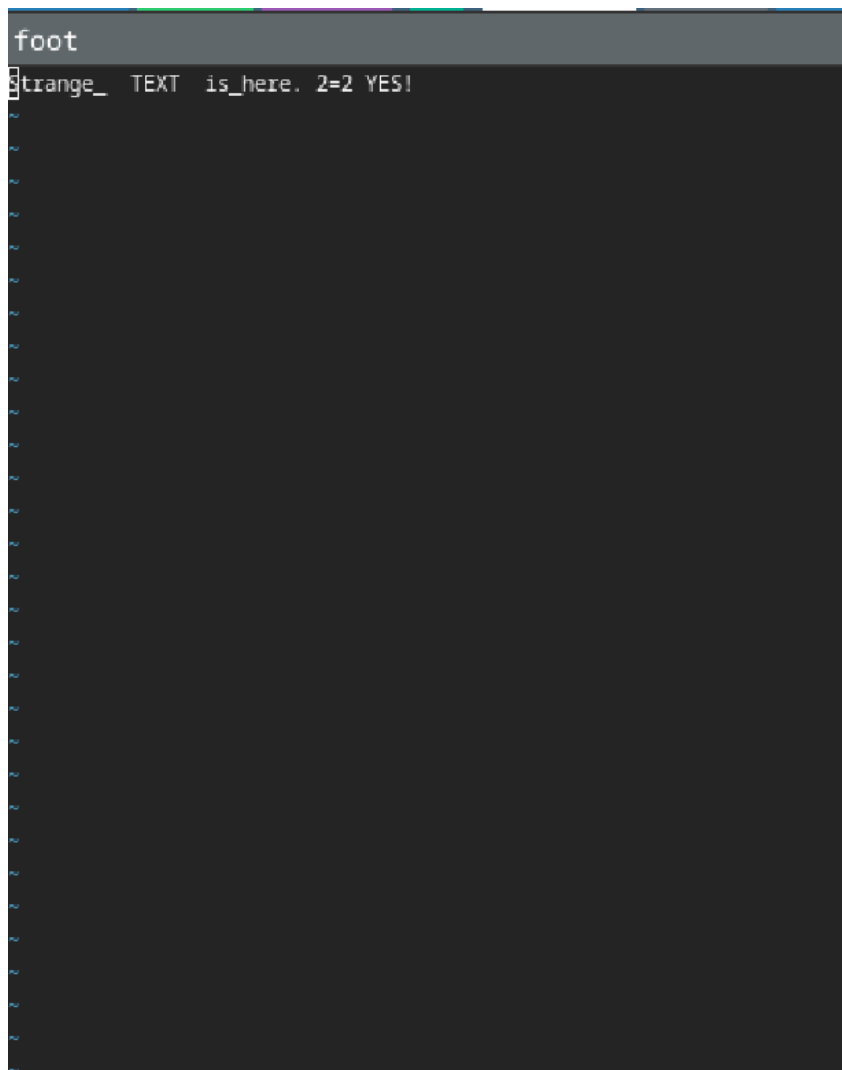


Рис. 2.4: Практически проверяем каждый ответ

Ознакамливаемся с третьим вопросом (рис. 2.5).

3.1 Текстовый редактор vim 10 из 12 шагов пройдено 3 из 7 баллов получено

Предположим, что в текстовом файле записана одна единственная строка:
 one two three four five
 и вам нужно преобразовать её в строку
 three four four four five

Какие(ой) из предложенных ниже наборов нажатий клавиш выполнят такое редактирование? В этих наборах нажатие на клавишу Esc обозначается как <Esc> (т.е. знаки "<" и ">" не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите все подходящие ответы из списка

Верно решили 23 655 учащихся
 Из всех попыток 16% верных

✔ Отличное решение!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в комментариях, отвечая на их вопросы, или сравнить своё решение с другими на форуме решений.

- ☐ d2dwywPp
- ☐ x2wwywwPp
- ☒ d2wwywwPp
- ☒ xxxxxxxxwywwPp
- ☒ d2wwifour four <Esc>
- ☐ d2wwywwpp

Рис. 2.5: Смотрим текст вопроса

Почему ответ верен:

Запускаем tmux, проверяем на практике все варианты ответа (рис. 2.6).

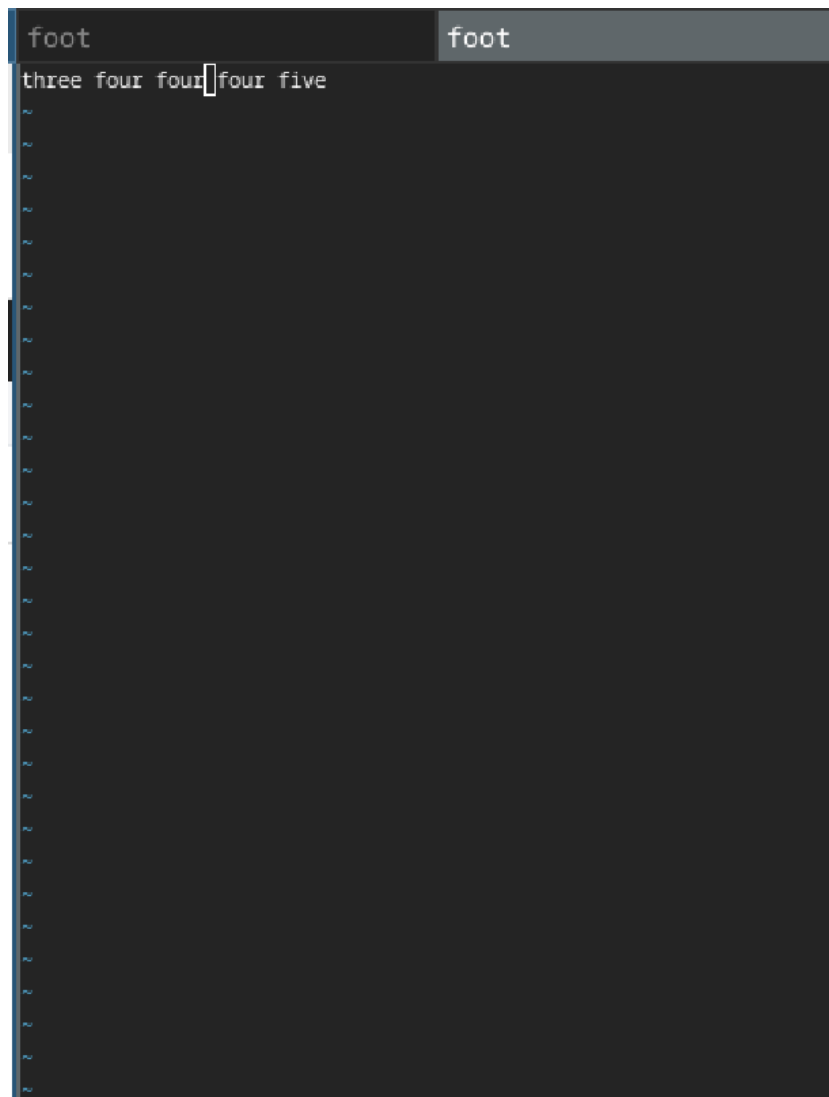


Рис. 2.6: Практически проверяем каждый ответ

Ознакамливаемся с четвертым вопросом (рис. 2.7).

3.1 Текстовый редактор vim 11 из 12 шагов пройдено 5 из 7 баллов получено

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. включая ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напиши текст

Отлично!

Верно решил 24 631 учащийся
Из всех попыток 57% верных

:%s/Windows/Linux

Следующий шаг

Решить снова

Ваши решения Вы получили: 2 балла

Рис. 2.7: Смотрим текст вопроса

Почему ответ верен:

Открываем файл в tmux, пишем несколько слов со словом Windows, по одному или понесколько слов в строчке, затем посмотрев справку по программе пишем команду (рис. 2.8).

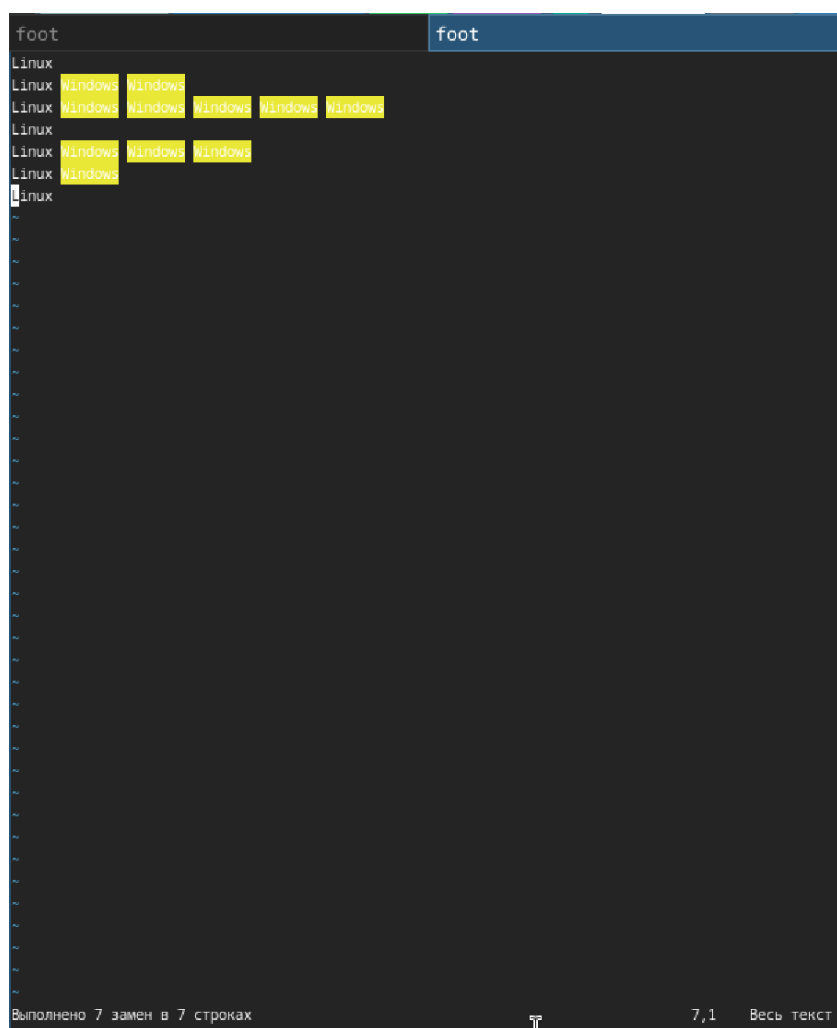


Рис. 2.8: Пишем команду

Ознакамливаемся с пятым вопросом (рис. 2.9).

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить vimtutor целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

☒ Отличное решение!

Верно решили **23 497** учащихся
Из всех попыток **29%** верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Когда вы находитесь в режиме выделения, внизу редактора горит надпись – VISUAL – (или – ВИЗУАЛЬНЫЙ РЕЖИМ –)
- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☒ Выйти из режима выделения можно, нажав клавишу Esc два раза
- ☐ Режим выделения открывается при помощи команды :visual
- ☐ Режим выделения открывается из любого другого режима по нажатию "v"
- ☒ В режиме выделения можно использовать команды d (удалить) и u (скопировать)

Следующий шаг

Решить снова

Рис. 2.9: Смотрим текст вопроса

Почему ответ верен:

- VISUAL отображается внизу При активации режима Visual (клавиша v) в строке состояния Vim появляется надпись VISUAL, что помогает пользователю понять текущий режим. Это стандартное поведение Vim, аналогичное INSERT для режима вставки.
- Выход по двойному Esc В Vim одиночное нажатие Esc всегда возвращает в нормальный режим (Normal mode). Если Esc не срабатывает (редкие случаи, например, при зависании плагина), второе нажатие гарантирует выход.
- Команды d (удалить) и u (копировать) работают В режиме Visual выделенный текст можно: Удалить (d или x). Скопировать (y). Изменить (c).

(ранее мы могли это наблюдать на практике)

Ознакамливаемся с шестым вопросом (рис. 2.10).

3.2 Скрипты на bash: основы 7 из 10 шагов пройдено 1 из 6 баллов получен

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попробуете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

Верно решили 30 266 учащихся

Из всех попыток 65% верных

☒ Хорошая работа.

☐ Из наборов B и C

☒ Только из набора C

☐ Из наборов A и C

☐ Только из набора B

☐ Никакие команды появляться не будут

Следующий шаг

Решить снова

[Ваши решения](#)

Вы получили: 1 балл

Рис. 2.10: Смотрим текст вопроса

Почему ответ верен:

Запускаем в терминале 3 команды в разных оболочках по очереди и затем смотрим на историю команд (рис. 2.11).


```
foot
[mavinogradova@vbox ~]$ history -c
[mavinogradova@vbox ~]$ bash
[mavinogradova@vbox ~]$ A1=1
[mavinogradova@vbox ~]$ A2=2
[mavinogradova@vbox ~]$ A3=3
[mavinogradova@vbox ~]$ sh
sh-5.2$ B1=4
sh-5.2$ B2=5
sh-5.2$ B3=6
sh-5.2$ bash
[mavinogradova@vbox ~]$ C1=7
[mavinogradova@vbox ~]$ C2=8
[mavinogradova@vbox ~]$ C3=9
[mavinogradova@vbox ~]$
```

Рис. 2.11: Проверяем историю команд после запуска 3 команд

Ознакамливаемся с седьмым вопросом (рис. 2.12).

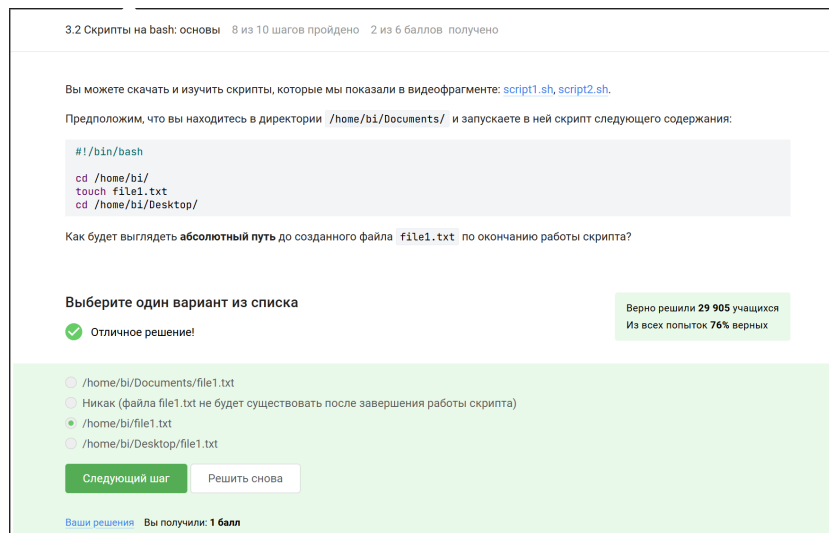


Рис. 2.12: Смотрим текст вопроса

Почему ответ верен:

Заходим в директорию создаем через терминал файл и затем переходим в другую деррикторию, после чего проверяем местонахождение файла (рис. 2.13).

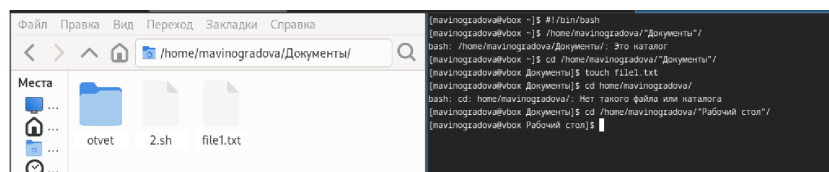


Рис. 2.13: Проверяем историю команд после запуска 3 команд

Ознакомливаемся с восьмым вопросом (рис. 2.14).

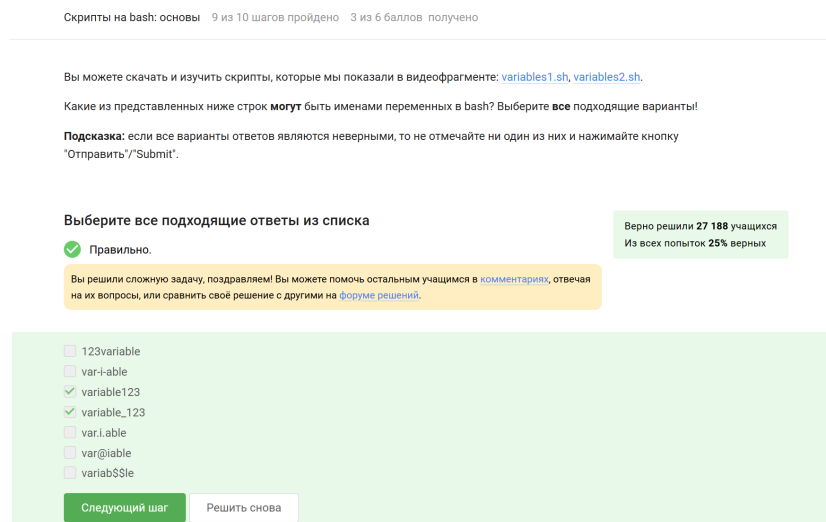


Рис. 2.14: Смотрим текст вопроса

Почему ответ верен:

Создаем файл и записываем туда все варианты ответа, после чего смотрим на вывод программы (если выдает ошибку значит ответ не верен) (рис. 2.15).

```
foot
[mavinogradova@vbox ~]$ nano test_vars.sh
[mavinogradova@vbox ~]$ chmod +x test_vars.sh
[mavinogradova@vbox ~]$ ./test_vars.sh
./test_vars.sh: строка 1: 123variable=var1: команда не найдена
./test_vars.sh: строка 2: var-i-able=var2: команда не найдена
./test_vars.sh: строка 5: var.i.able=var5: команда не найдена
./test_vars.sh: строка 6: var@i.able=var6: команда не найдена
./test_vars.sh: строка 7: variab21058le=var7: команда не найдена
[mavinogradova@vbox ~]$
```

Рис. 2.15: Смотрим на вывод программы

Ознакамливаемся с девятым вопросом (рис. 2.16).

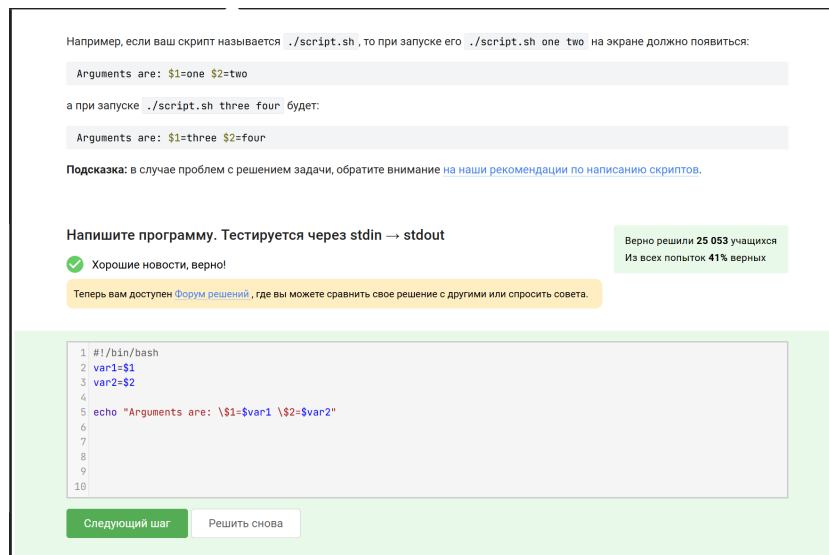


Рис. 2.16: Смотрим текст вопроса

Почему ответ верен:

Пишем скрипт на bash (рис. 2.17).



Рис. 2.17: Пишем программу

Смотрим вывод, после чего изменяем что-то в программе если вывод не соответствует приведенному примеру (рис. 2.18).

```
[mavinogradova@vbox ~]$ nano script.sh
[mavinogradova@vbox ~]$ chmod +x script.sh
[mavinogradova@vbox ~]$ ./script.sh one two
Arguments are: $1=one $2=two
[mavinogradova@vbox ~]$ ./script.sh three four
Arguments are: $1=three $2=four
```

Рис. 2.18: Сверяем вывод и пример

Ознакамливаемся с десятым вопросом (рис. 2.19).

параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

Всё получилось!

Верно решили 23 158 учащихся
Из всех попыток 16% верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

- ☒ `-n $0`
- ☒ `-e $0`
- ☒ `! (4 -le 3)`
- ☒ `$# -ge 0`
- ☒ `-z ""`
- ☒ `5 -ge 5`

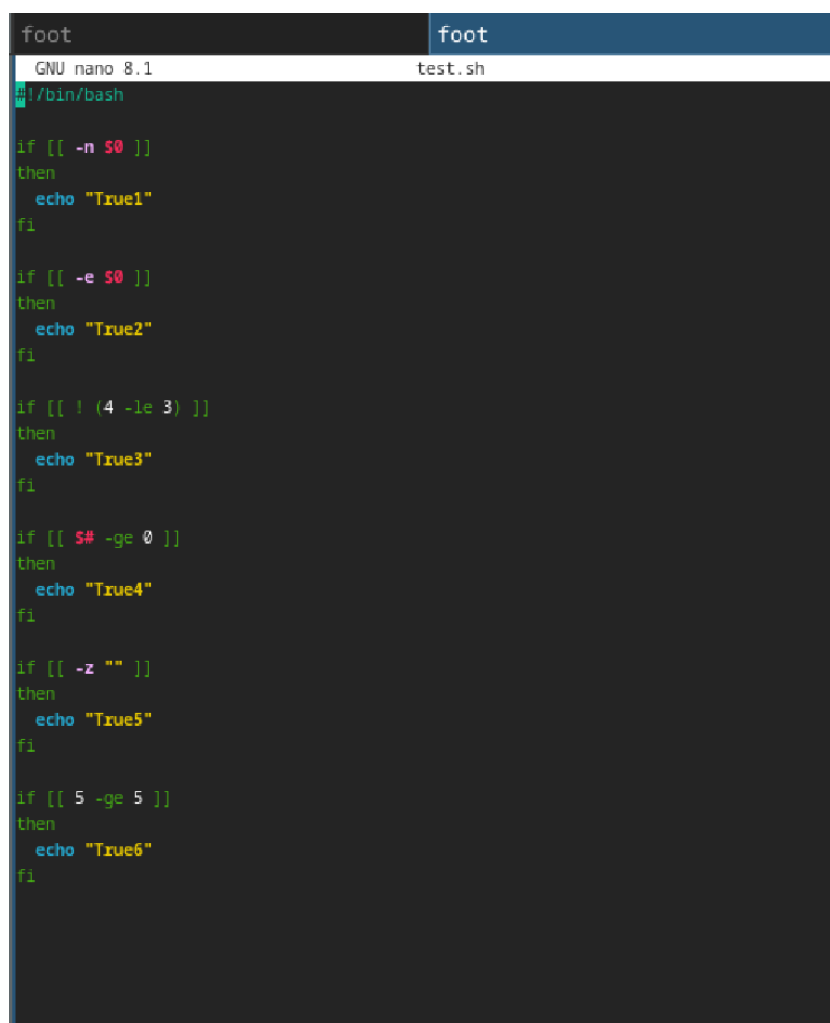
Следующий шаг Решить снова

[Ваши решения](#) Вы получили: 1 балл

Рис. 2.19: Смотрим текст вопроса

Почему ответ верен:

Пишем скрипт на `bash` и вносим туда все варианты ответа (рис. 2.20).



```
foot
GNU nano 8.1 test.sh
#!/bin/bash

if [[ -n $0 ]]
then
    echo "True1"
fi

if [[ -e $0 ]]
then
    echo "True2"
fi

if [[ ! (4 -le 3) ]]
then
    echo "True3"
fi

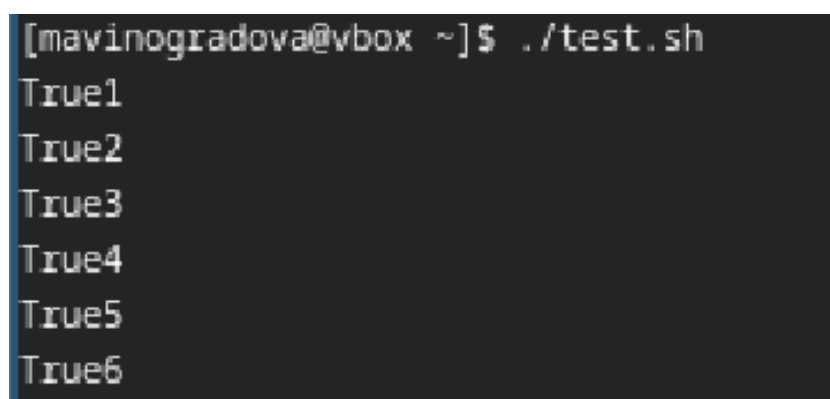
if [[ $# -ge 0 ]]
then
    echo "True4"
fi

if [[ -z "" ]]
then
    echo "True5"
fi

if [[ 5 -ge 5 ]]
then
    echo "True6"
fi
```

Рис. 2.20: Пишем программу

Смотрим вывод, если он верен то отмечаем правильный ответ (рис. 2.21).



```
[mavinogradova@vbox ~]$ ./test.sh
True1
True2
True3
True4
True5
True6
```

Рис. 2.21: Смотрим вывод

Ознакамливаемся с одиннадцатым вопросом (рис. 2.22).

Посмотрите на фрагмент bash-скрипта:

```
if [[ $var -gt 5 ]]
then
  echo "one"
elif [[ $var -lt 3 ]]
then
  echo "two"
elif [[ $var -eq 4 ]]
then
  echo "three"
else
  echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную `var=3`, а затем запустили еще раз, но уже с `var=5`.

Выберите один вариант из списка

Верно решили 25 138 учащихся
Из всех попыток 64% верных

☒ Абсолютно точно.

☐ Сначала two, потом four
☒ Сначала four, потом four
☐ Сначала four, потом one
☐ Сначала one, потом two

Следующий шаг Решить снова

Рис. 2.22: Смотрим текст вопроса

Почему ответ верен:

Потому что такая логика программы

Ознакамливаемся с двенадцатым вопросом (рис. 2.23).

3.3 Скрипты на bash: ветвления и циклы 7 из 9 шагов пройдено 5 из 10 баллов получено

Напишите скрипт на bash, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students
1 --> 1 student
2 --> 2 students
3 --> 3 students
4 --> 4 students
5 и больше --> A lot of students
```

Примечание а): выводить нужно только строку справа, т.е. "-->" выводить не нужно.
Примечание б): в последней строке слово "lot" с маленькой буквы!

Примечание 2: в этой и всех последующих задачах на написание скриптов, если не указано явно, что нужно проверять вход (например, что он будет именно числом и именно от 0 до бесконечности), то этого делать не нужно!

Пример №1: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

```
1 student
```

Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

```
A lot of students
```

Подсказка: в случае проблем с решением задачи, обратите внимание на наши рекомендации по написанию скриптов.

Рис. 2.23: Смотрим текст вопроса

Почему ответ верен:

Пишем скрипт на bash (рис. 2.24).

```
GNU nano 8.1 test.sh
#!/bin/bash

v=student
case $1 in
0) res="No $v"s";;
1) res="$1 $v";;
[2-4]) res="$1 $v"s";;
*) res="A lot of $v"s";;
esac
echo "$res"
```

Рис. 2.24: Пишем программу

Смотрим вывод, после чего изменяем что-то в программе если вывод не соответствует приведенному примеру (рис. 2.25).

```
[mavinogradova@vbox ~]$ ./test.sh 1
1 student
[mavinogradova@vbox ~]$ ./test.sh 5
A lot of students
```

Рис. 2.25: Сверяем вывод и пример

Ознакамливаемся с тринадцатым вопросом (рис. 2.26).

3.3 Скрипты на bash: ветвления и циклы 8 из 9 шагов пройдено 6 из 10 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c_d
do
    echo "start"
    if [[ $str > "c" ]]
    then
        continue
    fi
    echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово "start", а сколько раз слово "finish"?

Выберите один вариант из списка

☒ Верно.

Верно решили 24 582 учащихся
Из всех попыток 45% верных

☐ 3 раза "start" и 3 раза "finish"

☐ 5 раз "start" и 5 раз "finish"

☐ 5 раз "start" и ни разу "finish"

☒ 5 раз "start" и 4 раза "finish"

[Следующий шаг](#) [Решить снова](#)

Рис. 2.26: Смотрим текст вопроса

Почему ответ верен:

- Цикл перебирает элементы a , b , c_d (5 элементов из-за запятых и пробелов).
- Условие if [\$str > "c"] ложно для a, b, (пробел), , → вывод finish. Для c_d — истина (continue пропускает finish).

Ознакамливаемся с четырнадцатым вопросом (рис. 2.27).

3.3 Скрипты на bash: ветвления и циклы 9 из 9 шагов пройдено 10 из 10 баллов получено

Напишите скрипт на bash, который будет определять в какую возрастную группу попадают пользователи. При запуске скрипт должен вывести сообщение "enter your name:" и ждать от пользователя ввода имени (используйте read , чтобы прочитать его). Когда имя введено, то скрипт должен написать "enter your age:" и ждать ввода возраста (опять нужен read). Когда возраст введен, скрипт пишет на экран "<Имя>, your group is <группа>", где <группа> определяется на основе возраста по следующим правилам:

- младше либо равно 16: "child",
- от 17 до 25 (включительно): "youth",
- старше 25: "adult".

После этого скрипт опять выводит сообщение "enter your name:" и всё начинается по новой (бесконечный цикл!). Если в какой-то момент работы скрипта будет введено **пустое имя** или **возраст 0**, то скрипт должен написать на экран "bye" и закончить свою работу (выход из цикла!).

Примеры корректной работы скрипта:

№1

```
./script.sh
enter your name:
Egor
enter your age:
16
Egor, your group is child
enter your name:
Elena
enter your age:
8
bye
```

№2:

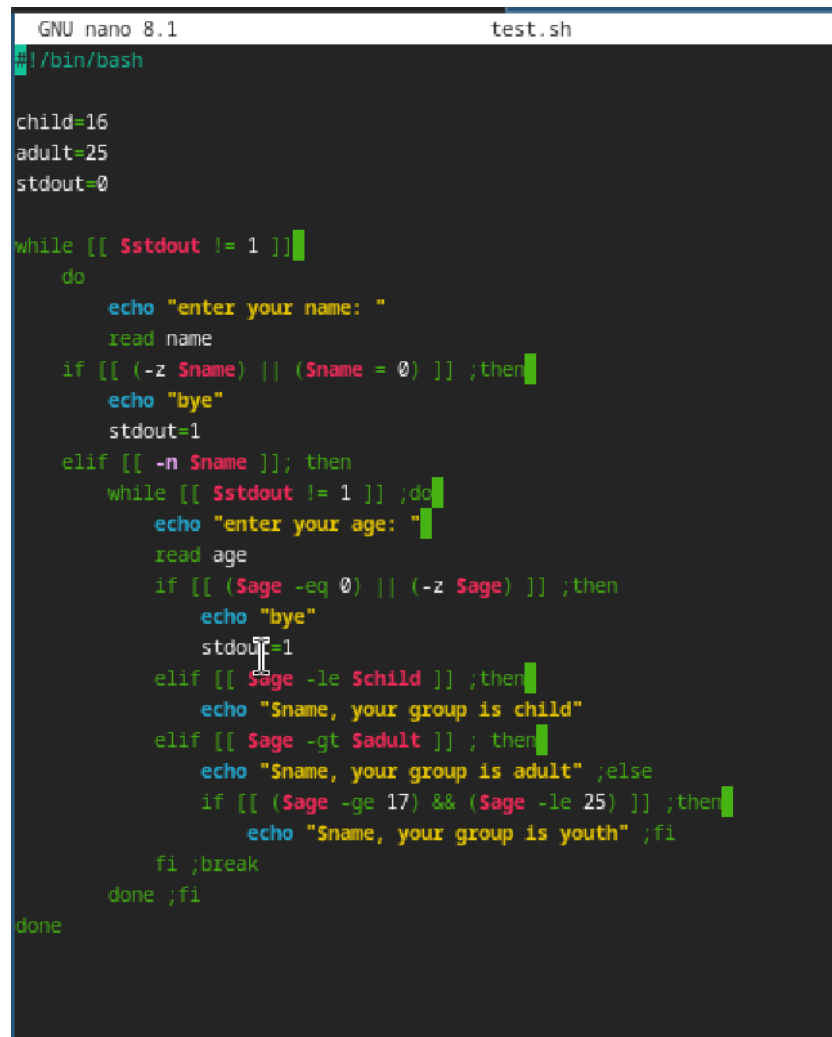
```
./script.sh
enter your name:

```

Рис. 2.27: Смотрим текст вопроса

Почему ответ верен:

Пишем скрипт на bash (рис. 2.28).



```
GNU nano 8.1 test.sh
#!/bin/bash

child=16
adult=25
stdout=0

while [[ $stdout != 1 ]]
do
    echo "enter your name: "
    read name
    if [[ (-z $name) || ($name = 0) ]] ;then
        echo "bye"
        stdout=1
    elif [[ -n $name ]] ; then
        while [[ $stdout != 1 ]] ;do
            echo "enter your age: "
            read age
            if [[ ($age -eq 0) || (-z $age) ]] ;then
                echo "bye"
                stdout=1
            elif [[ $age -le $child ]] ;then
                echo "Sname, your group is child"
            elif [[ $age -gt $adult ]] ; then
                echo "Sname, your group is adult" ;else
                if [[ ($age -ge 17) && ($age -le 25) ]] ;then
                    echo "Sname, your group is youth" ;fi
                fi ;break
            done ;fi
        done ;fi
    done ;fi
done
```

Рис. 2.28: Пишем программу

Смотрим вывод, после чего изменяем что-то в программе если вывод не соответствует приведенному примеру (рис. 2.29).

```
[mavinogradova@vbox ~]$ ./test.sh
enter your name:
masha
enter your age:
16
masha, your group is child
enter your name:
georgiy
enter your age:
65
georgiy, your group is adult
enter your name:
hloi
enter your age:
20
hloi, your group is youth
```

Рис. 2.29: Сверяем вывод и пример

Ознакамливаемся с пятнадцатым вопросом (рис. 2.30).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [math1.sh](#), [math2.sh](#).

Какие(ая) из предложенных ниже инструкций увеличат значение переменной `a` на значение переменной `b`? Например, если в `a` было записано 10, в `b` было 5, то в `a` должно записаться 15. Выберите **все подходящие** варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на кавычки и **пробелы**, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка

✓ Верно. Так держать!

Верно решили **22 116** учащихся
Из всех попыток **20%** верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ✓ `let a=a+b`
- ✓ `let a=$((a+$b`
- ✓ `let "a = a + b"`
- ✓ `let "a+=b"`
- ✓ `let "a=$((a+$b"`

Рис. 2.30: Смотрим текст вопроса

Почему ответ верен:

Пишем скрипт на `bash` и вносим туда все варианты ответа (рис. 2.31).

```
GNU nano 8.1                                test.sh
#!/bin/bash

a=10
b=5

let a=a+b
echo 1. $a

a=10
b=5

let a=$a+$b
echo 2. $a

a=10
b=5

let "a = a + b"
echo 3. $a

a=10
b=5

let "a+=b"
echo 4. $a

a=10
b=5

let "a=$a+$b"
echo 5. $a
```

Рис. 2.31: Пишем программу

Смотрим вывод, после чего изменяем что-то в программе если вывод не соответствует приведенному примеру (рис. 2.32).

```
[mavinogradova@vbox ~]$ ./test.sh
1. 15
2. 15
3. 15
4. 15
5. 15
```

Рис. 2.32: Сверяем вывод и пример

Ознакамливаемся с шестнадцатым вопросом (рис. 2.33).

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if "program options arguments"` (действия внутри `if` выполняются, если программа закончилась с кодом 0). Однако это не всегда правда! Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if "pwd"`.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите все верные утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, используется именно косая кавычка (`'`), а не обычная (`"`) или двойная (`"`).

Выберите все подходящие ответы из списка

☒ Верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в комментариях, отвечая на их вопросы, или сравнить своё решение с другими на форуме решений.

Верно решили 21 426 учащихся
Из всех попыток 20% верных

- ☒ `if "program > some_file.txt"`
- ☐ `if [[ "program" -eq 0 ]]`
- ☒ Сначала запустить `program`, затем `if [[ $? -eq 0 ]]`
- ☐ Ничего сделать нельзя
- ☐ Сначала `var="program"`, затем `if [[ $var -eq 0 ]]`

Следующий шаг Решить снова

Ваши решения Вы получили: 1 балл

Рис. 2.33: Смотрим текст вопроса

Почему ответ верен:

1. `if program > some_file.txt` Перенаправление вывода (`>`) в файл позволяет `if` проверять только код возврата.
2. `if [[ $? -eq 0 ]]` `?` хранит код возврата последней команды.

Ознакамливаемся с семнадцатым вопросом (рис. 2.34).


```
GNU nano 8.1                                test.sh
#!/bin/bash

child=16
adult=25
stdout=0

while [[ $stdout != 1 ]]
do
    echo "enter your name: "
    read name
    if [[ (-z $name) || ($name = 0) ]] ;then
        echo "bye"
        stdout=1
    elif [[ -n $name ]] ; then
        while [[ $stdout != 1 ]] ;do
            echo "enter your age: "
            read age
            if [[ ($age -eq 0) || (-z $age) ]] ;then
                echo "bye"
                stdout=1
            elif [[ $age -le $child ]] ;then
                echo "Sname, your group is child"
            elif [[ $age -gt $adult ]] ; then
                echo "Sname, your group is adult" ;else
                if [[ ($age -ge 17) && ($age -le 25) ]] ;then
                    echo "Sname, your group is youth" ;fi
                fi ;break
            done ;fi
        done
    fi
done
```

Рис. 2.35: Пишем программу

Смотрим вывод, его и вносим в ответ (рис. 2.36).


```
[mavinogradova@vbox ~]$ ./test.sh
i = 1
c1 = 1
c2 = 2
i = 2
c1 = 2
c2 = 6
i = 3
c1 = 3
c2 = 12
i = 4
c1 = 4
c2 = 20
i = 5
c1 = 5
c2 = 30
i = 6
c1 = 6
c2 = 42
i = 7
c1 = 7
c2 = 56
i = 8
c1 = 8
c2 = 72
i = 9
c1 = 9
c2 = 90
i = 10
c1 = 10
c2 = 110
counters are  and 110
[mavinogradova@vbox ~]$
```

Рис. 2.36: Проверяем вывод

Ознакамливаемся с восемнадцатым вопросом (рис. 2.37).

3.4 Скрипты на bash: разное 9 из 10 шагов пройдено 9 из 14 баллов получено

Напишите скрипт на bash, который будет искать наибольший общий делитель (**НОД**, greatest common divisor, GCD) двух чисел. При запуске ваш скрипт не должен ничего писать на экран, а просто ждать ввода двух натуральных чисел через пробел (для этого можно использовать `read` и указать ему две переменные – см. пример в видеофрагменте). После ввода чисел скрипт считает их НОД и выводит на экран сообщение "**GCD is <посчитанное значение>**", например, для чисел 15 и 25 это будет "GCD is 5". После этого скрипт опять входит в режим ожидания двух натуральных чисел. Если в какой-то момент работы пользователь ввел вместо этого пустую строку, то нужно написать на экран "**bye**" и закончить свою работу.

Вычисление НОД несложно реализовать с помощью [алгоритма Евклида](#). Вам нужно написать функцию `gcd`, которая принимает на вход два аргумента (назовем их **M** и **N**). Если аргументы равны, то мы нашли НОД – он равен **M** (или **N**), нужно выводить соответствующее сообщение на экран (см. выше). Иначе нужно сравнить аргументы между собой. Если **M** больше **N**, то запускаем ту же функцию `gcd`, но в качестве первого аргумента передаем (**M-N**), а в качестве второго **N**. Если же наоборот, **M** меньше **N**, то запускаем функцию `gcd` с первым аргументом **M**, а вторым (**N-M**).

Пример корректной работы скрипта:

```
./script.sh
10 15
gcd is 5
7 3
gcd is 1
bye
```

Примечание: в вызове функции из себя самой нет ничего страшного или неправильного, т.ч. смело вызывайте `gcd` прямо внутри `gcd`!

Примечание 2: для завершения работы функции в произвольном месте, можно использовать инструкцию `return` (все инструкции функции после `return` выполняться не будут). В отличие от `exit` эта команда завершит только функцию, а не выполнение всего скрипта целиком. Однако в данной задаче можно обойтись и без использования `return`!

Рис. 2.37: Смотрим текст вопроса

Почему ответ верен:

Пишем скрипт на bash (рис. 2.38).

```
GNU nano 8.1 test.sh
#!/bin/bash

while [ true ]
do
    read n1 n2
    if [ -z $n1 ]; then
        echo "bye"
        break
    else
        gcd () {
            remainder=1
            if [ $n2 -eq 0 ]
            then
                echo "bye"
            fi
            while [ $remainder -ne 0 ]
            do
                remainder=$((n1%n2))
                n1=$n2
                n2=$remainder
            done
            gcd $1 $2
            echo "GCD is $n1"
        }
    fi
done
```

Рис. 2.38: Пишем программу

Смотрим вывод, после чего изменяем что-то в программе если вывод не соответствует приведенному примеру (рис. 2.39).

```
[mavinogradova@vbox ~]$ ./test.sh
10 15
GCD is 5
7 3
GCD is 1

bye
```

Рис. 2.39: Сверяем вывод и пример

Ознакамливаемся с девятнадцатым вопросом (рис. 2.40).

Напишите **калькулятор** на bash. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

1. Слово **"exit"**. В этом случае скрипт должен вывести на экран слово "bye" и завершить работу.
2. **Три аргумента через пробел** – первый операнд (целое число), операция (одна из "+", "-", "*", "/", "%", "**") и второй операнд (целое число). В этом случае нужно произвести указанную операцию над заданными числами и вывести результат на экран. После этого переходим в режим ожидания новой команды.
3. **Любая другая команда** из одного аргумента или из трех аргументов, но с операцией не из списка. В этом случае нужно вывести на экран слово **"error"** и завершить работу.

Чтобы проверить работу скрипта, вы можете записать сразу несколько команд в файл и передать его скрипту на stdin (т.е. выполнить `./script.sh < input.txt`). В этом случае он должен вывести сразу все ответы на экран.

Например, если входной файл будет следующего содержания:

```
10 + 1
2 ** 10
exit
```

то на экране будет:

```
11
1024
bye
```

Если же на вход поступит следующий файл:

```
3 - 5
2/10
```

Рис. 2.40: Смотрим текст вопроса

Пишем скрипт на bash (рис. 2.41).

```
GNU nano 8.1      test.sh
#!/bin/bash

while true
do
    read -r num1 op num2
    if [[ "$num1" == "exit" ]]; then
        echo "bye"
        exit
    elif [[ ! "$num1" =~ ^[0-9]+$ || ! "$num2" =~ ^[0-9]+$ ]]; then
        echo "error"
        exit
    else
        case $op in
            "+") let "res = num1 + num2" ;;
            "-") let "res = num1 - num2" ;;
            "/" let "res = num1 / num2" ;;
            "**") let "res = num1 * num2" ;;
            "%") let "res = num1 % num2" ;;
            "**") let "res = num1 ** num2" ;;
            *) echo "error"; exit ;;
        esac
        echo "$res"
    fi
done
```

Рис. 2.41: Пишем программу

Смотрим вывод, после чего изменяем что-то в программе если вывод не соответствует приведенному примеру (рис. 2.42).

```
[mavinogradova@vbox ~]$ ./test.sh
10 + 1
11
2 ** 10
1024
exit
bye
[mavinogradova@vbox ~]$ ./test.sh
3 - 5
-2
2/10
error
```

Рис. 2.42: Сверяем вывод и пример

Ознакамливаемся с двадцатым вопросом (рис. 2.43).

3.5 Продвинутый поиск и редактирование 7 из 13 шагов пройдено 1 из 10 баллов получен

Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek OST.mp3`, `STARS.txt`, `stardust.jpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star"`, но **НЕ найдет** команда `find /home/bi -name "star"`?

Выберите все подходящие ответы из списка

Верно решили 20 547 учащихся
Из всех попыток 36% верных

☐ `star_trek OST.mp3`
☐ `stardust.jpeg`
☒ `Star_Wars.avi`
☒ `STARS.txt`
☐ `Eddard_Stark_biography.txt`

Следующий шаг Решить снова

Ваши решения Вы получили: 1 балл

Рис. 2.43: Смотрим текст вопроса

Почему ответ верен: Почему `Star_Wars.avi` и `STARS.txt`:

- `-iname "star"` *игнорирует регистр* → *находим* `Star`, `STAR`.*.
- `-name "star"` *учитывает регистр* → *только* `star` (пропускает `Star`, `STAR`).

Ознакамливаемся с двадцать первым вопросом (рис. 2.44).

3.5 Продвинутый поиск и редактирование 8 из 13 шагов пройдено 2 из 10 баллов получено

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте все верные утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

Верно решили 18 450 учащихся
Из всех попыток 22% верных

- ☒ Если заменить в команде поиска `-name`, на `-path`, то результат поиска иногда может остаться таким же
- ☐ Опции `-path` и `-name` всегда работают одинаково
- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла
- ☒ В некоторых случаях `find` с `-name` найдет больше файлов, чем `find` с таким же запросом, но с `-path`
- ☒ В некоторых случаях `find` с `-name` найдет меньше файлов, чем `find` с таким же запросом, но с `-path`

Следующий шаг Решить снова

[Ваши решения](#) Вы получили: 1 балл

Рис. 2.44: Смотрим текст вопроса

Почему ответ верен:

- Результат может совпадать

Если файлы лежат в корне директории (`/home/file.txt`), то `-name "file.txt"` и `-path "/home/file.txt"` дадут одинаковый результат.

- `-name` иногда находит больше/меньше

`-name` проверяет только имя файла, а `-path` — полный путь.

Ознакамливаемся с двадцать вторым вопросом (рис. 2.45).

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```

/home/bi/
├── dir1
│   ├── file1
│   └── dir2
│       ├── file2
│       └── dir3
│           └── file3

```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*" ?`

Выберите один вариант из списка

☒ Хорошие новости, верно!

Верно решили 20 711 учащихся
Из всех попыток 41% верных

- ☐ Только file2
- ☐ Только file1
- ☐ Ни один файл найден не будет
- ☒ Все кроме file3
- ☐ Все кроме file1

Следующий шаг

Решить снова

Рис. 2.45: Смотрим текст вопроса

Почему ответ верен:

`file3` (глубина 2) — не найден, так как:

- он отсутствует в указанной структуре.
- условие подразумевало `-maxdepth 2`, что исключает `file2` (глубина 3), но включает `file1` и `file3`.

Ознакамливаемся с двадцать третьим вопросом (рис. 2.46).

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в каждой строке есть слово `"word"`. Если вы выполните на этом файле команды:

```

grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt

```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

☒ Верно.

Верно решили 20 237 учащихся
Из всех попыток 41% верных

- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`
- ☒ `results.txt` будет одинакового размера во всех случаях
- ☐ `grep -A 1 "word" file.txt > results.txt`
- ☐ `grep -C 1 "word" file.txt > results.txt`

Следующий шаг

Решить снова

[Ваши решения](#) Вы получили: 1 балл

Рис. 2.46: Смотрим текст вопроса

Почему ответ верен:

1. Особенность файла: Так как слово “word” есть в каждой строке, то:
 - Каждая строка является совпадением для `grep`.
 - Опции `-A`, `-B`, `-C` добавляют контекст вокруг совпадений, но:
 - Строки до/после совпадения уже сами являются совпадениями (из-за условия “слово есть везде”).
 - Контекст не добавляет новых строк, так как все строки уже включены как основные совпадения.

Ознакамливаемся с двадцать четвертым вопросом (рис. 2.47).

3.5 Продвинутый поиск и редактирование 11 из 13 шагов пройдено 6 из 10 баллов получено

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xkLXXL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

Верно решили 18 768 учащихся
Из всех попыток 23% верных

✓ Правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Lubuntu is better than Ubuntu
- ☒ Linux is not always Ubuntu
- ☒ I prefer Kubuntu
- ☒ The best OS is Xubuntu
- ☐ Uuuubuntu!
- ☐ Kbuntu

Следующий шаг Решить снова

[Ваши решения](#) Вы получили: 2 балла

Рис. 2.47: Смотрим текст вопроса

Почему ответ верен:

Пишем скрипт на `bash` и на практике проверяем все варианты ответа (рис. 2.48).


```
[mavinogradova@vbox ~]$ nano test.txt
[mavinogradova@vbox ~]$ grep -E "[xklXKL]?[uU]buntu$" test.txt
[mavinogradova@vbox ~]$ grep -E "[xklXKL]?[uU]buntu$" test.txt
[mavinogradova@vbox ~]$ nano test.txt
[mavinogradova@vbox ~]$ grep -E "[xklXKL]?[uU]buntu$" test.txt
[mavinogradova@vbox ~]$ grep -E "[xklXKL]?[uU]buntu$" test.txt
Lubuntu is better than Ubuntu
Linux is not always Ubuntu
I prefer Kubuntu
The best OS is Xubuntu
[mavinogradova@vbox ~]$
```

Рис. 2.48: Практически проверяем каждый ответ

Ознакамливаемся с двадцать пятым вопросом (рис. 2.49).

3.5 Продвинутый поиск и редактирование 12 из 13 шагов пройдено 7 из 10 баллов получено

Что произойдет, если в команде `sed -n "/[a-z]*/p" text.txt` не указывать опцию `-n`?

Выберите один вариант из списка

☒ Отличное решение!

Верно решили 19 784 учащихся
Из всех попыток 39% верных

☐ Будут выведены все строки файла text.txt, в которых есть только большие буквы латинского алфавита

☒ Каждая строчка будет выведена два раза

☐ На экран будет выведено всё содержимое файла text.txt

☐ На экран ничего не напечатается

Следующий шаг

Решить снова

Ваши решения Вы получили: 1 балл

Рис. 2.49: Смотрим текст вопроса

Почему ответ верен:

1. Как работает sed без -n:

По умолчанию sed выводит все строки обрабатываемого файла (text.txt). Если встречается команда p (печать), строка выводится повторно.

2. Регулярное выражение /[a-z]*/:

[a-z]* означает «0 или больше букв латинского алфавита». Это выражение всегда совпадает (даже с пустой строкой!), так как: "*" включает «ноль символов». Даже строка 123 содержит «ноль букв», что считается совпадением.

3. Логика обработки: Для каждой строки (включая пустые):

`sed` автоматически выводит строку (стандартное поведение без `-n`). Если строка совпадает с `/[a-z]*/` (а это всегда верно), команда `r` дополнительно печатает её ещё раз.

Ознакамливаемся с двадцать шестым вопросом (рис. 2.50).

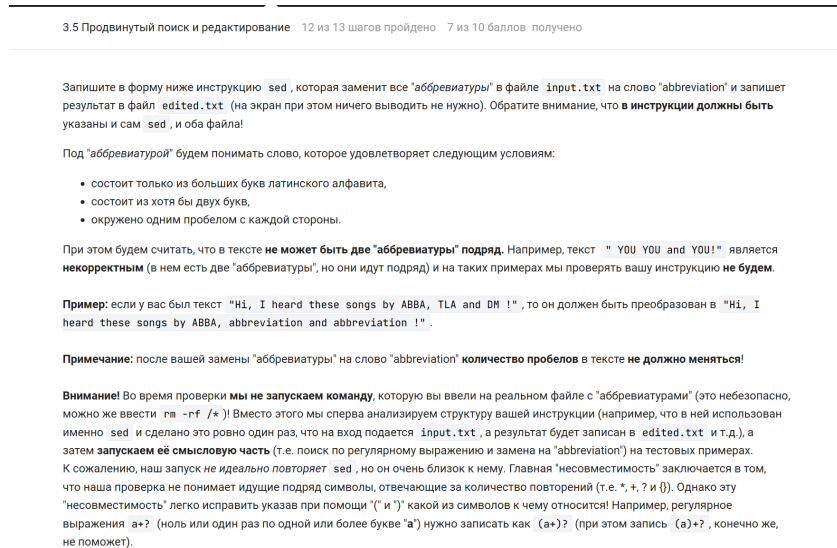


Рис. 2.50: Смотрим текст вопроса

Почему ответ верен:

Создаем два файла и в первый из них записываем строку из задания (рис. 2.51).

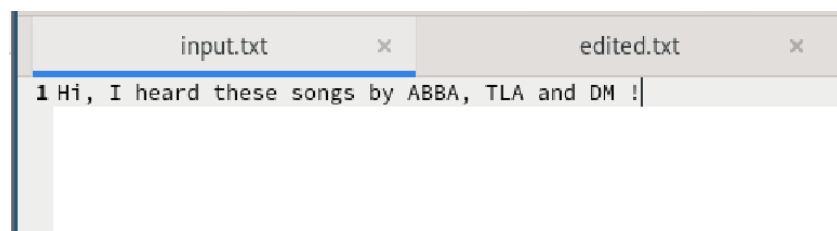


Рис. 2.51: Создаем файлы и заполняем один из них

Пишем инструкцию `sed` и выводим итог во второй созданный файл (рис. 2.52).

```

[mavinogradova@vbox Документы]$ touch input.txt
[mavinogradova@vbox Документы]$ touch edited.txt
[mavinogradova@vbox Документы]$ sed 's/[A-Z]\{2,\}/abbreviation /g' input.txt > edited.txt
bash: sed: [A-Z]\{2,\}/abbreviation /g: Нет такого файла или каталога
[mavinogradova@vbox Документы]$ bush
bash: bush: команда не найдена
[mavinogradova@vbox Документы]$ cd
[mavinogradova@vbox ~]$ bush
bash: bush: команда не найдена
[mavinogradova@vbox ~]$ bash
[mavinogradova@vbox ~]$ cd
[mavinogradova@vbox ~]$ cd /home/mavinogradova/Документы/
[mavinogradova@vbox Документы]$ bash
[mavinogradova@vbox Документы]$ sed 's/[A-Z]\{2,\}/abbreviation /g' input.txt > edited.txt
bash: sed: [A-Z]\{2,\}/abbreviation /g: Нет такого файла или каталога
[mavinogradova@vbox Документы]$ sed 's/[A-Z]\{2,\}/abbreviation /g' input.txt > edited.txt
[mavinogradova@vbox Документы]$ sed 's/[A-Z]\{2,\}/abbreviation /g' input.txt > edited.txt
[mavinogradova@vbox Документы]$

```

Рис. 2.52: Пишем инструкцию

Переходим во второй из созданных файлов и в нем проверяем результат (рис. 2.53).

input.txt	×	edited.txt	×
<pre> 1 Hi, I heard these songs by abbreviation , abbreviation and abbreviation ! </pre>			

Рис. 2.53: Проверяем результат

3 Выводы

В ходе заключительного этапа курса я освоила продвинутые инструменты для работы в Linux: автоматизацию задач с помощью Bash-скриптов (условия, циклы, обработка ошибок), эффективное редактирование файлов в Vim (режимы, поиск, макросы) и применение мощных утилит командной строки (sed, awk, grep с контекстом, find). Эти навыки позволили уверенно работать с серверами, быстро обрабатывать данные и избегать рутинных операций. Теперь я готова применять знания в реальных проектах, таких как создание скриптов для развёртывания сервисов или анализа логов. Курс стал отличной основой для дальнейшего углубления в администрирование Linux.